Guía del alumno

Módulo 3, Limpieza y análisis exploratorio

Ruta guiada

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía acompaña al Lab 03 guiado y sigue sus dos partes en el mismo orden y con los mismos nombres del cuaderno. La Parte 1, En clase, tiene seis pasos, de Paso 1 de 6 a Paso 6 de 6, y se hace en clase. La Parte 2, Para estudiar, tiene tres estudios, de Estudio 1 de 3 a Estudio 3 de 3, y se hace después de clase con esta guía, una vez corrida la Parte 1. Cada paso trae el código y la respuesta tal como aparecen en Colab, y una explicación en palabras simples.

En los pasos no hay que escribir código. Lo que cambia se cambia en los deslizadores y en la lista desplegable, y después se vuelve a correr la celda. El lab responde tres preguntas. Qué hay que mirar en un archivo antes de calcularle nada, qué hacer con las filas repetidas, los datos que faltan y los valores imposibles, y qué tan bien quedó la limpieza.

Lo que vas a poder hacer al terminar

- Contar las filas de un archivo y compararlas con las que debería traer.
- Reconocer un número o una fecha que llegaron como texto, y convertirlos.
- Unificar las formas distintas de escribir una misma región.
- Quitar filas repetidas y decidir qué hacer con los datos que faltan.
- Encontrar valores imposibles con una regla de negocio.
- Saber cuándo el promedio no describe bien a un grupo.

Todas las respuestas de esta guía vienen de una corrida del cuaderno con los valores que trae por defecto. Los datos del curso son sintéticos, inventados para aprender, y no se citan como datos del sector.

Parte 1 · En clase

Seis pasos, uno detrás del otro, que se hacen en clase en el mismo orden del cuaderno.

Paso 1 de 6 · ¿Cómo llegó el archivo?

Antes de calcular nada se mira cómo llegó el archivo. Este es un archivo de demanda de cada hora de 2024 en las siete regiones, tal como llegaría desde una planilla, con errores.

Paso 1 de 6

```
# Paso 1
DATOS = ("https://raw.githubusercontent.com/RodrigoMSB/"
         "curso-python-minergia/main/datos/")
import pandas as pd

# El archivo viene separado por punto y coma, como lo guarda Excel en Chile.
sucia = pd.read_csv(DATOS + "demanda_sucia.csv", sep=";")

# 2024 fue bisiesto. 366 días por 24 horas por 7 regiones.
esperadas = 366 * 24 * 7
print("Filas del archivo ", len(sucia))
print("Filas esperadas   ", esperadas)
print("Sobran            ", len(sucia) - esperadas)
sucia.head()
```

```
Filas del archivo  61508
Filas esperadas    61488
Sobran             20
        fecha  hora       region    mwh
0  01/01/2024     0  Antofagasta  51,95
1  01/01/2024     1  Antofagasta  49,02
2  01/01/2024     2  Antofagasta  48,22
3  01/01/2024     3  Antofagasta  45,57
4  01/01/2024     4  Antofagasta  47,59
```

El archivo viene separado por punto y coma en vez de coma, como lo guarda Excel en Chile. Por eso la lectura lleva `sep=";"`.

La primera pregunta es si trae las filas que debería. 2024 fue bisiesto, así que son 366 días por 24 horas por 7 regiones, 61488 filas. El archivo trae 61508, sobran 20. Todavía no sabemos por qué, eso lo responde el Paso 3.

Mirando las primeras filas ya se ve algo raro. La demanda aparece con coma decimal, `51,95`, y la fecha con el día primero, `01/01/2024`.

Paso 2 de 6 · ¿Cómo se ordena el desorden?

Ahora se arreglan tres problemas del archivo, el número, la fecha y la región. La celda trabaja sobre una copia, así el archivo original queda intacto para comparar.

Paso 2 de 6

```
# Paso 2
limpia = sucia.copy()

# La coma decimal pasa a punto y el texto pasa a número.
limpia["mwh"] = limpia["mwh"].str.replace(",", ".").astype(float)

# El formato dice cómo viene escrita la fecha, día, mes y año.
limpia["fecha"] = pd.to_datetime(limpia["fecha"], format="%d/%m/%Y")

# strip quita los espacios de los extremos y title deja la primera letra en mayúscula.
limpia["region"] = limpia["region"].str.strip().str.title()

print("✓ Regiones distintas", limpia["region"].nunique())
print("✓ Demanda total del año", round(limpia["mwh"].sum()), "MWh")
print("✓ Fechas del", limpia["fecha"].min().date(), "al", limpia["fecha"].max().date())
```

```
✓ Regiones distintas 7
✓ Demanda total del año 13560560 MWh
✓ Fechas del 2024-01-01 al 2024-12-31
```

Cada línea arregla una cosa. La demanda cambia la coma por un punto y pasa de texto a número. La fecha se convierte con el formato día, mes y año. La región pierde los espacios de los extremos y queda con la primera letra en mayúscula, así `biobío` y `BIOBÍO` pasan a ser `Biobío`.

Las tres líneas con ✓ confirman el resultado. Quedan 7 regiones, la demanda total del año suma 13560560 MWh y las fechas van del 1 de enero al 31 de diciembre de 2024.

Paso 3 de 6 · ¿Hay filas repetidas?

En el Paso 1 sobraban 20 filas. Esta celda revisa si son copias.

Paso 3 de 6

```
# Paso 3
# duplicated marca cada fila idéntica a una que ya apareció antes.
print("Filas repetidas", limpia.duplicated().sum())

sin_repetidas = limpia.drop_duplicates()
print("Filas que quedan", len(sin_repetidas))
if len(sin_repetidas) == esperadas:
    print("✓ Ahora calza con las", esperadas, "filas esperadas")
```

```
Filas repetidas 20
Filas que quedan 61488
✓ Ahora calza con las 61488 filas esperadas
```

`duplicated` marca cada fila idéntica a otra que ya apareció antes. Hay 20, justo las que sobraban. `drop_duplicates` las quita y quedan 61488 filas, las que debía traer el archivo.

Las filas repetidas se quitan antes de sumar. Si no, la demanda de esas horas se cuenta dos veces y el total queda más alto de lo que fue.

Paso 4 de 6 · Tu turno, decide qué hacer con los datos que faltan

Quedan las horas sin dato. Hay varias formas de tratarlas, y cada una cambia el resultado.

Paso 4 de 6

```
# Paso 4
estrategia = "trazar una recta" #@param ["borrar la fila", "poner cero", "repetir el valor
↪  anterior", "trazar una recta"]

# Ordenadas por región y por hora, para que cada hora quede junto a sus vecinas.
orden = sin_repetidas.sort_values(["region", "fecha", "hora"]).reset_index(drop=True)
por_region = orden.groupby("region")["mwh"]

if estrategia == "borrar la fila":
    rellena = orden["mwh"].dropna()
elif estrategia == "poner cero":
    rellena = orden["mwh"].fillna(0)
elif estrategia == "repetir el valor anterior":
    rellena = por_region.ffill()
else:
    rellena = por_region.transform(lambda s: s.interpolate())

print(f"Promedio por hora con {estrategia}", round(rellena.mean(), 1), "MWh")

# Un día de Valparaíso que venía con cuatro horas vacías.
dia = orden[(orden["region"] == "Valparaíso") & (orden["fecha"] == "2024-09-16")]
vista = pd.DataFrame({"hora": dia["hora"],
                      "venía": dia["mwh"],
                      "queda": rellena.reindex(dia.index)})
vista.head(12).round(1).fillna({"venía": "vacía", "queda": "se borra"}).set_index("hora")
```

```
Promedio por hora con trazar una recta 227.5 MWh
      venía  queda
hora
0     177.0  177.0
1     vacía  173.3
2     vacía  169.7
3     166.0  166.0
4     164.7  164.7
5     179.2  179.2
6     191.4  191.4
7     222.5  222.5
8     vacía  230.8
9     vacía  239.1
10    247.4  247.4
11    259.6  259.6
```

Borrar la fila deja fuera las horas vacías, poner cero las llena con cero, repetir el valor anterior copia la hora de antes y trazar una recta rellena con los valores entre la hora anterior y la siguiente. Las dos últimas trabajan región por región.

Con trazar una recta el promedio por hora queda en 227.5 MWh. Con borrar la fila queda en 227.3, con poner cero baja a 220.5 y con repetir el valor anterior queda en 227.6. La tabla es un día de Valparaíso que venía con cuatro horas vacías. Con la recta, la hora 1 queda en 173.3 y la hora 8 en 230.8, entre sus vecinas. Elige otra estrategia y mira cómo cambian esas horas.

> LA DECISIÓN QUE NO SE VE EN EL INFORME

> Las cuatro estrategias dan resultados distintos con los mismos datos. Poner cero baja el promedio y repetir el valor anterior aplana el día. Ninguna es la correcta siempre, pero la que se use se deja escrita. Desde el próximo paso seguimos con trazar una recta.

Paso 5 de 6 · Tu turno, decide qué valor es imposible

Algunas horas traen un valor que no puede ser real. Se encuentran con reglas de negocio, que son reglas que salen de conocer el tema.

Paso 5 de 6

```
# Paso 5
veces = 3 #@param {type:"slider", min:1, max:10, step:1}

tabla = sin_repetidas.sort_values(["region", "fecha", "hora"]).reset_index(drop=True)
promedio_region = tabla.groupby("region")["mwh"].transform("mean")

# Dos reglas de negocio. La demanda no puede ser negativa,
# y un valor muchas veces mayor que el promedio de su región es sospechoso.
negativo = tabla["mwh"] < 0
pico = tabla["mwh"] > veces * promedio_region
print("Horas con demanda negativa", negativo.sum())
print(f"Horas con más de {veces} veces el promedio de su región", pico.sum())

# Antes de tocar nada se marca qué horas se van a cambiar y por qué.
final = tabla.copy()
final["fue_nulo"] = final["mwh"].isna()
final["fue_corregido"] = negativo | pico

# Los valores imposibles se vacían y todos los huecos se rellenan con una recta.
final.loc[final["fue_corregido"], "mwh"] = None
final["mwh"] = final.groupby("region")["mwh"].transform(lambda s: s.interpolate())

print("✓ Horas rellenadas porque venían vacías", final["fue_nulo"].sum())
print("✓ Horas corregidas porque eran imposibles", final["fue_corregido"].sum())
print("✓ Horas sin dato que quedan", final["mwh"].isna().sum())
```

```
Horas con demanda negativa 15
Horas con más de 3 veces el promedio de su región 40
✓ Horas rellenadas porque venían vacías 1844
✓ Horas corregidas porque eran imposibles 55
✓ Horas sin dato que quedan 0
```

La celda usa dos reglas. La demanda no puede ser negativa, y hay 15 horas así. Y un valor muchas veces mayor que el promedio de su región es sospechoso. Con el deslizador en 3, hay 40 horas con más de tres veces el promedio de su región.

Antes de cambiar nada, la celda marca qué horas se van a tocar y por qué. La columna `fue_nulo` marca las horas que venían vacías y la columna `fue_corregido` marca las que eran imposibles. Después vacía las imposibles y rellena todos los huecos con una recta. Quedan 1844 horas rellenadas, 55 corregidas y ninguna sin dato.

El archivo trae 1846 horas vacías, como muestra el Estudio 1, y aquí son 1844. Las dos que faltan estaban en filas repetidas que se quitaron en el Paso 3.

Mueve el deslizador a 1 y vuelve a correr la celda. Marca 27964 horas, casi la mitad del año, porque muchas horas normales superan el promedio. Con 8 marca solo 33. En el Paso 6 se ve qué tan bien quedó cada elección.

Paso 6 de 6 · ¿Qué tan bien quedó la limpieza?

Este lab tiene algo que el trabajo real no tiene, el archivo original sin errores. Así se puede medir cuánto se equivocó la limpieza.

Paso 6 de 6

```
# Paso 6
# Este lab tiene algo que el trabajo real no tiene, el archivo original sin errores.
verdad = pd.read_csv(DATOS + "demanda.csv", parse_dates=["fecha"])

auditoria = pd.merge(final, verdad, on=["fecha", "hora", "region"],
                     suffixes=("_limpia", "_real"))
auditoria["error"] = (auditoria["mwh_limpia"] - auditoria["mwh_real"]).abs()

auditoria["grupo"] = "no se tocó"
auditoria.loc[auditoria["fue_nulo"], "grupo"] = "se rellenó"
auditoria.loc[auditoria["fue_corregido"], "grupo"] = "se corrigió"

print("Horas comparadas", len(auditoria))
auditoria.groupby("grupo")["error"].agg(horas="count", error_promedio="mean",
                                        error_maximo="max").round(1)
```

```
Horas comparadas 61488
             horas  error_promedio  error_maximo
grupo
no se tocó   59589             0.0           0.0
se corrigió     55             4.6          48.0
se rellenó    1844             5.4          66.0
```

La celda une la tabla limpia con la original, hora por hora, y calcula el error en cada hora, la diferencia entre los dos valores sin importar el signo. Se comparan las 61488 horas.

Después separa las horas en tres grupos, con las marcas del Paso 5. Las 59589 horas que no se tocaron tienen error cero. Las 1844 que se rellenaron se equivocan 5.4 MWh en promedio, con un máximo de 66.0. Las 55 que se corrigieron se equivocan 4.6 en promedio, con un máximo de 48.0. El error está solo donde se intervino, y se sabe dónde gracias a las marcas.

Si en el Paso 5 pones el deslizador en 8 y corres ese paso y después este, el grupo de horas que no se tocaron deja de tener error cero. Su error máximo sube a 6036.4, porque con un corte tan alto quedan sin corregir valores que eran imposibles.

> **ESTO NO SE PUEDE HACER EN EL TRABAJO REAL**
>
> Aquí se pudo medir el error porque existía el archivo original. En el trabajo real el dato sucio es el único que hay. Lo que sí se puede hacer siempre es marcar qué horas se rellenaron y cuáles se corrigieron, como hacen fue_nulo y fue_corregido.

Parte 2 · Para estudiar

Estos pasos se hacen después de clase, con esta guía. Antes de empezar hay que correr toda la Parte 1 del cuaderno, porque los estudios usan sus tablas.

Estudio 1 de 3 · ¿Qué tiene de raro?

Cuatro preguntas rápidas que dicen qué problemas trae el archivo.

Estudio 1 de 3

```
# Estudio 1
print("Regiones distintas     ", sucia["region"].nunique(), " y deberían ser 7")
print("Horas sin dato de mwh  ", sucia["mwh"].isna().sum())

# repr muestra el valor con comillas cuando es texto.
print("Primer valor de mwh    ", repr(sucia["mwh"].iloc[0]), " es texto, no número")
print("Primera fecha          ", repr(sucia["fecha"].iloc[0]), " es texto, no fecha")
```

```
Regiones distintas      49  y deberían ser 7
Horas sin dato de mwh   1846
Primer valor de mwh     '51,95'  es texto, no número
Primera fecha           '01/01/2024'  es texto, no fecha
```

Hay 49 formas distintas de escribir las regiones, cuando deberían ser 7. Hay 1846 horas sin dato de demanda.

Y hay dos problemas que no se ven a simple vista. La demanda, `'51,95'`, y la fecha, `'01/01/2024'`, aparecen entre comillas. Eso quiere decir que Python las leyó como texto. Con un texto no se puede sumar ni pedir el mes, así que hay que convertirlos antes de hacer cualquier cálculo.

Estudio 2 de 3 · Tu turno, mira cómo está escrita cada región

Las 49 formas de escribir las regiones, contadas de la más frecuente a la menos frecuente.

Estudio 2 de 3

```
# Estudio 2
cuantas = 12 #@param {type:"slider", min:5, max:49, step:1}

# value_counts cuenta cuántas veces aparece cada forma, de mayor a menor.
sucia["region"].value_counts().head(cuantas)
```

```
region
Valparaíso       8714
Antofagasta      8713
Coquimbo         8707
Metropolitana    8698
Atacama          8693
Los Lagos        8686
Biobío           8683
biobío             26
los lagos          21
BIOBÍO             21
metropolitana      20
atacama            19
Name: count, dtype: int64
```

`value_counts` cuenta cuántas veces aparece cada forma. Las siete primeras son las regiones bien escritas, con más de 8000 horas cada una. Desde la octava aparecen las mal escritas, en minúscula, en mayúscula o con espacios. La octava es `biobío`, con 26 horas.

Mueve el deslizador para ver más filas y vuelve a correr la celda. Si una región mal escrita se queda así, cualquier resumen por región la cuenta como si fuera otra región.

Estudio 3 de 3 · ¿Cuánto consume una región en una hora normal?

Con los datos ya limpios se puede describir la demanda. La pregunta es si el promedio describe bien una hora típica.

Estudio 3 de 3

```
# Estudio 3
# La mediana es el valor del medio. La mitad de las horas está por debajo y la otra mitad por
↪  encima.
print("Todo el país junto")
print("  Promedio", round(final["mwh"].mean(), 1), " Mediana", round(final["mwh"].median(), 1))

resumen = final.groupby("region")["mwh"].agg(promedio="mean", mediana="median")
resumen.round(1).sort_values("promedio", ascending=False)
```

```
Todo el país junto
  Promedio 226.1  Mediana 111.5
              promedio  mediana
region
Metropolitana     885.7    873.4
Valparaíso        207.9    204.6
Biobío            185.9    183.1
Los Lagos          98.4     97.1
Coquimbo           93.0     91.6
Antofagasta        76.6     75.5
Atacama            35.0     34.5
```

La **mediana** es el valor del medio. La mitad de las horas está por debajo y la otra mitad por encima.

Con todas las regiones juntas, el promedio es 226.1 MWh y la mediana 111.5, menos de la mitad. Pero dentro de cada región los dos números casi coinciden. En la Metropolitana el promedio es 885.7 y la mediana 873.4. En Atacama, 35.0 y 34.5.

> **EL PROMEDIO DE TODO EL PAÍS NO DESCRIBE A NINGUNA REGIÓN**
>
> Con todas las regiones juntas el promedio es el doble de la mediana. No es por valores extremos, que ya se corrigieron. Es porque la Metropolitana consume varias veces más que las demás. Dentro de cada región el promedio y la mediana están casi iguales.

Para seguir practicando

Al final del cuaderno hay cuatro ejercicios para quien quiera seguir después de clase. En ellos sí se escribe código. Antes de empezar hay que correr todo el cuaderno con Entorno de ejecución, Ejecutar todo, porque los ejercicios usan las tablas de los pasos.

Cada ejercicio tiene una celda vacía para escribir la respuesta y una solución plegada, que conviene abrir solo después de intentarlo. Las soluciones no están en esta guía.

- **Ejercicio 1.** Arreglar el archivo en la misma lectura, con la coma decimal.
- **Ejercicio 2.** Comparar si es mejor rellenar con el promedio de la región y la hora que trazar una recta.
- **Ejercicio 3.** Calcular la demanda alta de cada región con el percentil 95.
- **Ejercicio 4.** Armar una tabla con la demanda promedio por región y por franja del día.

Para repasar

Referencia rápida

Lo que se escribe	Para qué sirve
`pd.read_csv(..., sep=";")`	Lee un archivo separado por punto y coma
`value_counts()`	Cuántas veces aparece cada valor
`str.replace(",", ".")`	Cambia la coma decimal por punto
`pd.to_datetime(..., format=...)`	Convierte un texto en fecha
`str.strip().str.title()`	Quita espacios y deja la primera letra en mayúscula
`duplicated()` · `drop_duplicates()`	Marca y quita filas repetidas
`fillna` · `ffill` · `interpolate`	Tres formas de rellenar huecos
`mean()` · `median()`	Promedio y mediana

Si algo sale mal

Lo que aparece en pantalla	Qué pasó
`NameError: name 'sucia' is not defined`	Corriste un paso sin correr antes el Paso 1, que es el que lee el archivo.
`NameError: name 'final' is not defined`	El Paso 6 y el Estudio 3 usan la tabla que arma el Paso 5. Corre el Paso 5 antes.
El Paso 6 no cambió después de mover el deslizador del Paso 5	Hay que correr el Paso 5 y después el Paso 6. Mover el deslizador no corre nada.
`ValueError: could not convert string to float`	Aparece si se convierte la demanda a número sin cambiar antes la coma por punto, que es lo que hace el Paso 2.

Tres preguntas de repaso

1. Por qué la demanda llegó como texto

Porque viene con coma decimal. Python solo entiende el punto como decimal, así que lee `51,95` como texto.

2. Por qué hay que dejar escrita la forma de rellenar los huecos

Porque cada forma da un resultado distinto con los mismos datos, y quien lea el informe tiene que saber cuál se usó.

3. Para qué sirven las columnas fue_nulo y fue_corregido

Para saber después qué horas se rellenaron y cuáles se corrigieron. Sin esas marcas, los valores inventados quedan escondidos entre los reales.

El Lab 03 guiado en una plana

Para imprimir y tener al lado durante el curso

Los pasos de las dos partes

N.	La pregunta	La idea
Parte 1, en clase		
1	¿Cómo llegó el archivo?	Contar las filas
2	¿Cómo se ordena el desorden?	Convertir número, fecha y región
3	¿Hay filas repetidas?	Quitar las copias
4	Tu turno, decide qué hacer con los datos que faltan	Cuatro formas de rellenar
5	Tu turno, decide qué valor es imposible	Reglas de negocio y marcas
6	¿Qué tan bien quedó la limpieza?	Medir el error por grupo
Parte 2, para estudiar		
1	¿Qué tiene de raro?	Texto donde debería haber números
2	Tu turno, mira cómo está escrita cada región	Una región escrita de muchas formas
3	¿Cuánto consume una región en una hora normal?	Promedio y mediana

El orden de una limpieza

Primero mirar	Contar filas, revisar tipos y valores distintos
Después convertir	Números, fechas y textos a su forma correcta
Después quitar copias	Antes de sumar nada
Marcar antes de cambiar	Qué horas se rellenan y cuáles se corrigen
Dejar escrito el criterio	La forma de rellenar y el corte de lo imposible

Los métodos estadísticos para encontrar valores extremos y el detalle de cada estrategia de relleno están en la ruta completa de este módulo.